

Topic 3B: How Transformers Compute

*Last updated: September 24, 2026**Surbhi Goel*

Acknowledgements: The discussion of induction heads draws on Elhage et al. [3], Olsson et al. [4], Bietti et al. [5], and Edelman et al. [6]. The depth and lower-bound discussion draws on Hsu's lectures [1, 2], Sanford, Hsu, and Telgarsky [8, 9], and Liu et al. [11]. The softmax estimate draws on Edelman et al. [7]. The broader discussion draws on the surveys of Liu, Makkua, and Lee [10] and Strobl et al. [15], and on Merrill and Sabharwal [12].

AI Disclosure: These notes were drafted and revised with AI assistance. If you find an error, please let me know.

In the last lecture, we defined the Transformer architecture. Here we ask what it can compute. Three related questions will guide us:

- **Representation:** Do weights exist that compute the function, and what resources do they require?
- **Learnability:** Can training find suitable weights from a specified distribution of examples?
- **Generalization:** On which new inputs or sequence lengths does the learned computation remain correct?

Most of these notes address representation. We will construct an induction head, compare the resources needed by one and two layers, and then study repeated lookups. The next lecture will address learning and generalization.

1 Induction Heads

An *induction head* implements a simple copying rule: when the current token is a , find an earlier occurrence of a and copy the token b that followed it to predict the next token [3]. If there are several earlier occurrences, we will use the most recent one whose continuation is already in the prompt.

Prompt

... a b ... a

Predicted next token

b

The association between a and b comes from the current prompt and can change from one input to another. We will study how a Transformer can implement this rule for arbitrary tokens a and b using the same weights.

Olsson et al. [4] found that induction heads’ emergence during training correlates with a sharp improvement in *in-context learning*: using information in the prompt without changing the model’s weights.¹ Figure 1 shows this correlation. Here performance is measured by how much prediction loss improves with more context. Removing these heads from small attention-only models removes much of that improvement.

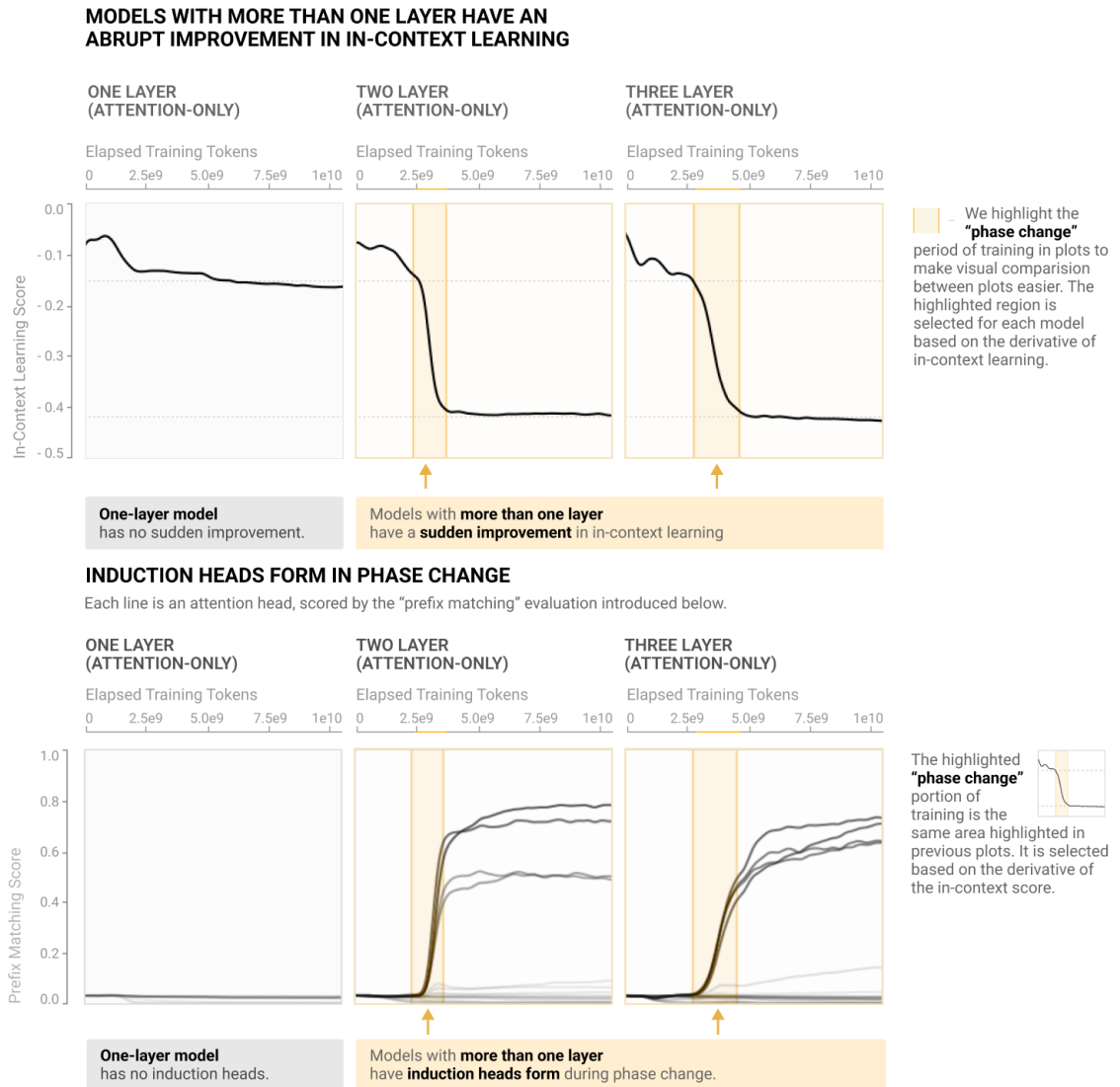


Figure 1: Training one-, two-, and three-layer attention-only models on text. Top: loss at token 500 minus loss at token 50. Lower values mean more benefit from context. Bottom: attention to the continuation of an earlier match. Shading marks the transition during training. The plots are from Olsson et al. [4]. Their [larger-model results](#) cover models with MLPs and up to 13 billion parameters.

¹For example, an English–French translation prompt might give `hello -> bonjour` and `thanks -> merci`, then end with `bread ->`. The intended continuation is `pain`: the demonstrations specify the task without a weight update.

2 A Construction of Induction Heads

Can we choose the query, key, and value maps to implement this rule for any input? We give one explicit construction to show that such weights exist. Other embeddings, attention maps, and intermediate representations could implement the same rule.

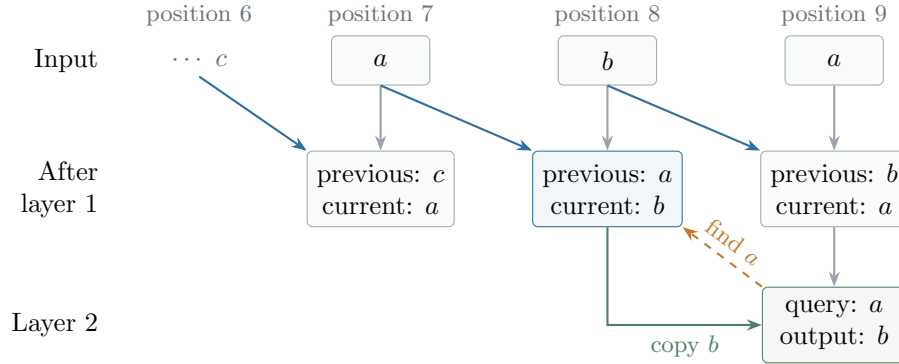
Consider the following sequence, with one token per letter:

Position	1	2	3	4	5	6	7	8	9
Token	<i>d</i>	<i>e</i>	<i>c</i>	<i>d</i>	<i>b</i>	<i>c</i>	<i>a</i>	<i>b</i>	<i>a</i>

The final *a* should retrieve *b* at position 8. To choose that position, we need to know that its predecessor was *a*. This information is initially at position 7. We therefore use two layers:

1. Copy the preceding token into a separate part of each position's representation. Position 8 now holds its current *b* and its predecessor *a*.
2. Match the final *a* against the copied predecessors, then read the current token at the matching position. Position 8 supplies the answer *b*.

The first layer prepares every pair in parallel. The second layer uses those pairs for retrieval. This two-head organization follows Bietti et al. [5] and Edelman et al. [6]. Below we give explicit embeddings and maps for the most-recent-match rule of Sanford, Hsu, and Telgarsky [8, 9].



What each position stores. Let $x_1, \dots, x_T \in \mathcal{V}$ be the input, with $T \geq 2$. Add a beginning token $x_0 = \perp$, with $\perp \notin \mathcal{V}$, which will also be the answer when no match exists. Let e_w be the one-hot vector for token w , including $\perp$. We store the current token in h_i^{cur} and the copied predecessor in h_i^{prev} , using separate coordinates for these two vectors. Initially,

$$z_i^{(0)} = (h_i^{\text{cur}}, h_i^{\text{prev}}, 1, i, i^2) = (e_{x_i}, 0, 1, i, i^2), \quad 0 \leq i \leq T.$$

The token and position embeddings also occupy separate coordinates, as in previous lectures. The constant coordinate lets a linear map add a fixed offset. The coordinates i, i^2 will let a query select a position using squared distance. Attention is causal, so position i reads positions $0, \dots, i$. We use one head and a pointwise ReLU MLP in each block, omit layer normalization, and absorb $1/\sqrt{d_{\text{head}}}$ into the query map.

We first describe attention as an exact lookup:

$$j^* = \arg \max_{0 \leq j \leq i} \mathbf{q}_i^\top \mathbf{k}_j, \quad o_i = v_{j^*}.$$

We will then account for the error from softmax.

First layer: copy the predecessor. Choose

$$\mathbf{q}_i = (2i - 2, -1), \quad \mathbf{k}_j = (j, j^2), \quad v_j = h_j^{\text{cur}}.$$

The query uses the stored i and the constant coordinate to form $2i - 2$. The key uses the source's j, j^2 , and the value reads its current token h_j^{cur} . Their score is

$$\mathbf{q}_i^\top \mathbf{k}_j = 2(i - 1)j - j^2 = (i - 1)^2 - (j - (i - 1))^2.$$

For $i \geq 1$, the square is smallest at $j = i - 1$, so the attention output is $e_{x_{i-1}}$. The output projection puts this vector in the coordinates for h_i^{prev} , which were initially zero, and writes zero elsewhere. Residual addition therefore gives

$$(h_i^{\text{cur}}, h_i^{\text{prev}}) = (e_{x_i}, 0) + (0, e_{x_{i-1}}) = (e_{x_i}, e_{x_{i-1}}).$$

Thus copying the predecessor preserves the current token. Position 0 can read only itself, so $h_0^{\text{cur}} = h_0^{\text{prev}} = e_\perp$.

Second layer: match and retrieve. The query reads h_i^{cur} , the key reads h_j^{prev} , and the value reads h_j^{cur} . Thus the token part of the score is

$$(h_i^{\text{cur}})^\top h_j^{\text{prev}} = \mathbf{1}\{x_i = x_{j-1}\} \quad (j \geq 1).$$

The final a matches the predecessor stored at position 8, whose value is e_b . The key determines which position to read, while the value determines what we copy from it. Token matching alone gives the same score to every match. To prefer the most recent one, add the source position j to the score. We must keep token matching more important than recency: a recent nonmatch should never beat an older match. Give each match a bonus of $T + 1$. Since $0 \leq j \leq T$, this bonus exceeds any possible advantage from recency.

We also need an answer when there is no match. Give position 0 a bonus of $T + 1$, so it beats every nonmatch but loses to every match. The full score is

$$s_{ij} = \underbrace{(T + 1)(h_i^{\text{cur}})^\top h_j^{\text{prev}}}_{\text{token match}} + \underbrace{j}_{\text{recency}} + \underbrace{(T + 1)h_{j,\perp}^{\text{cur}}}_{\text{default at position 0}}.$$

Here $h_{j,\perp}^{\text{cur}}$ is the $\perp$ coordinate of h_j^{cur} . It is 1 only at position 0. For an ordinary query token, the scores are

source	s_{ij}
nonmatch at $j \geq 1$	$j \leq T$
default at $j = 0$	$T + 1$
match at $j \geq 2$	$T + 1 + j$

Every match beats the default, which beats every nonmatch. Among two matches at $j < j'$, the score difference is $j' - j$, so the later one wins. The winning score is separated from every other score by at least 1.

For example, consider $a \ b \ a \ c \ a$, with $T = 5$. For the final a , positions 2 and 4 both store a as their predecessor. Their scores are $6 + 2 = 8$ and $6 + 4 = 10$, so we copy c from position 4. Position 5 is more recent but does not match, so its score is only 5. Position 0 has score 6. The value is always h_j^{cur} , including $h_0^{\text{cur}} = e_\perp$ when no match exists. Appendix A gives the linear projections and residual updates for these two heads.

From argmax to softmax. Softmax gives positive weight to every allowed position. The first layer therefore returns a mixture rather than an exact one-hot token for the second layer’s keys. Multiplying all scores by a positive number β concentrates attention on the largest score. How large must β be to make the copy accurate? The following bound is a special case of Edelman et al. [7, Lemma B.7]. For one query, write its scores as s_j and its weights as α_j .

Proposition 2.1 (Softmax selection). *Suppose a query attends to at most $T + 1$ positions. Let j^* have the largest score, separated from every other score by at least $\Delta > 0$. For any desired error $0 < \varepsilon < 1$, choose*

$$\beta \geq \frac{1}{\Delta} \log \frac{T}{\varepsilon}, \quad \alpha_j = \frac{\exp(\beta s_j)}{\sum_r \exp(\beta s_r)}.$$

Then $\alpha_{j^} \geq 1 - \varepsilon$. If each value coordinate lies in $[0, 1]$, the copied value satisfies*

$$\left\| \sum_j \alpha_j v_j - v_{j^*} \right\|_\infty \leq \varepsilon. \quad (1)$$

Proof. For each unwanted position j , cancel the common softmax denominator and use the score gap:

$$\frac{\alpha_j}{\alpha_{j^*}} = \exp(-\beta(s_{j^*} - s_j)) \leq e^{-\beta\Delta} \leq \frac{\varepsilon}{T}.$$

Since $\alpha_{j^*} \leq 1$, each such position receives at most ε/T attention. There are at most T of them, so

$$1 - \alpha_{j^*} = \sum_{j \neq j^*} \alpha_j \leq \varepsilon.$$

For the value bound, subtract v_{j^*} from the weighted average. Each coordinate of $v_j - v_{j^*}$ has absolute value at most 1, giving

$$\left\| \sum_j \alpha_j v_j - v_{j^*} \right\|_\infty = \left\| \sum_{j \neq j^*} \alpha_j (v_j - v_{j^*}) \right\|_\infty \leq \sum_{j \neq j^*} \alpha_j \leq \varepsilon. \quad \square$$

Both heads in our construction have $\Delta \geq 1$. Taking $\varepsilon = 1/16$ therefore requires only $\beta \geq \log(16T)$. For a one-hot value, the copied answer coordinate is at least 15/16 and every other coordinate is at most 1/16.

After the first attention step, a pointwise ReLU MLP recovers the exact predecessor by applying

$$R(t) = 2 \operatorname{ReLU}(t - 1/4) - 2 \operatorname{ReLU}(t - 3/4)$$

to h_i^{prev} . This function is 0 below 1/4 and 1 above 3/4. It sends the large answer coordinate to 1 and all other coordinates to 0. The second head therefore receives exact one-hot keys. Its output has the desired token as its largest coordinate. The appendix shows how to read that answer after residual addition using the same coordinates.

Width and precision. Each of h_i^{cur} and h_i^{prev} has one coordinate for every token, including $\perp$. With the three positional coordinates, the residual width is

$$d = 2(|\mathcal{V}| + 1) + 3 = O(|\mathcal{V}|).$$

This is the width of the explicit encoding above, not a lower bound on the width needed for induction heads. For a fixed vocabulary it is constant as T grows. One-hot vectors make token matching particularly simple: their dot product is exactly 1 for a match and 0 otherwise.

Simulating one attention layer. We use ordinary softmax attention for this lower bound. Suppose Alice and Bob both know the Transformer’s weights. The token at position T is always a , so they both know the final query $\mathbf{q}_T$. This query is independent of Bob’s index r . In one layer, each key and value depends only on its own input token and position. Alice can therefore compute the keys and values at her even positions without knowing Bob’s index r . Bob can compute them at his odd positions, including the final a , and at the fixed beginning token in position 0.

For one head, absorb the usual scaling into the query and write $w_j = \exp(\mathbf{q}_T^\top \mathbf{k}_j)$. Alice computes her part of the numerator and denominator:

$$A = \sum_{j=1}^s w_{2j} v_{2j}, \quad Z_A = \sum_{j=1}^s w_{2j}.$$

Bob computes the corresponding sums B and Z_B over the odd positions and position 0. The final attention output is

$$o_T = \frac{A + B}{Z_A + Z_B}.$$

Thus Alice only needs to send her two sums for each head. Bob can then compute every head’s output, apply the remaining position-wise maps at position T , and recover u_r . There is no need to send a separate value for every input position.

We must count the bits in this message. Let H be the number of heads, d bound the embedding and head dimensions, and p be the bits of precision per coordinate, including positional information. Alice can communicate her contribution using $O(dp)$ bits per head.² Her total message uses $O(Hdp)$ bits. But INDEX requires at least s bits, so

$$Hdp = \Omega(s) = \Omega(T).$$

This proves the one-layer lower bound, even with arbitrary position-wise computation and an arbitrary output decoder.

Comparing one and two layers. For a fixed vocabulary and inputs of length T , our two-layer construction uses $d = O(1)$ and $p = O(\log T)$, so $Hdp = O(\log T)$. A one-layer model with the same precision must instead have $Hd = \Omega(T/\log T)$. Encoding positions in a few logarithmic-precision coordinates does not remove the separation.

Why does the second layer help? After the first layer of our construction, each even position holds both Alice’s token and the preceding marker supplied by Bob. Its second-layer key therefore depends on Bob’s index. Alice can no longer compute her part of the attention output without knowing that index, so the one-message simulation above no longer applies.

4 Repeated Lookups with Induction Heads

A single induction-head lookup uses one association in the prompt. A simple model of deeper reasoning within the context is to follow a chain of these associations: retrieve b from a , use that b to retrieve c , and continue. How many layers are needed for k successive lookups?

Use the same input as in Section 2:

²We use the finite-precision model of Sanford, Hsu, and Telgarsky [8]: rounding embeddings and attention outputs to p -bit representations must preserve the predicted tokens. To avoid sending large exponential sums, Alice sends A/Z_A and $\log Z_A$, each rounded to $O(p)$ bits per coordinate. Appendix A of that paper shows that Bob can recover the head output to the required precision.

Position	1	2	3	4	5	6	7	8	9
Token	d	e	c	d	b	c	a	b	a

The final a retrieves b at position 8. At that position, the earlier b at position 5 is followed by c at position 6. Repeating the rule gives

$$\underbrace{9}_a \mapsto \underbrace{8}_b \mapsto \underbrace{6}_c \mapsto \underbrace{4}_d \mapsto \underbrace{2}_e.$$

At each step, the query uses the prefix ending at the position just retrieved. Thus the b at position 8 can read positions $1, \dots, 8$, including the earlier bc pair. The ordering of this example makes every link available to causal attention.

The task. Write $f(i)$ for the position returned by one induction-head lookup. The k -hop induction-head task follows this rule k times and returns the token at the final destination, $x_{f^k(i)}$. Here f^k means composing f with itself k times. If a lookup finds no match, we return $\perp$ and keep that answer on all later steps. For simplicity, take k to be a power of two.

One lookup at a time. A direct solution follows the chain one step per layer. First, use the two-layer construction to find $f(i)$ at every position. Store both this destination and its token. Keep these one-step answers available. At position i , start with destination $f(i)$. A new layer reads the one-step answer at that destination, obtaining $f^2(i)$. The next reads the one-step answer there, obtaining $f^3(i)$. Continuing this way takes $k - 1$ additional layers, for total depth $k + 1$. Every layer extends the chain by just one lookup.

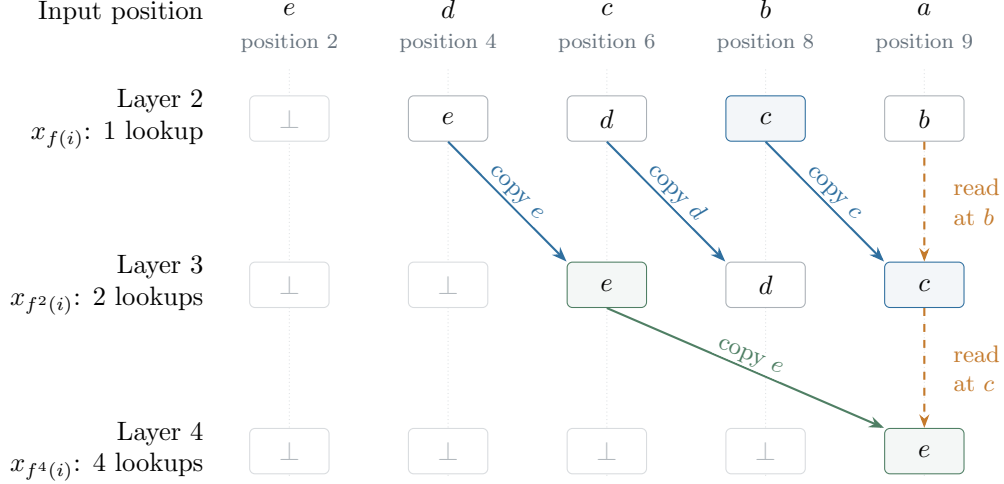
Doubling the number of lookups. We can do better by updating the stored answers at every position. Suppose each position i already stores its destination $f^s(i)$ after s lookups. Use this as the address to read: position $j = f^s(i)$ already stores $f^s(j)$. Copying that value gives

$$f^{2s}(i) = f^s(f^s(i)).$$

All positions read from the previous layer, so these updates happen in parallel. Starting from the one-step answers, successive layers therefore compute f^2 , f^4 , f^8 , and so on.³

In the diagram, each column is a fixed input position. We use its input token as shorthand, so column b means position 8. Each box shows the token at the stored destination. The model also stores the destination’s position, which tells the next layer where to read. Solid arrows show the copied answers. The dashed arrows track which destination column a uses at each layer. Copies of $\perp$ are omitted.

³Liu et al. [11] use function composition to simulate finite-state machines. Sanford et al. [9] give logarithmic-depth constructions for the k -hop induction-head task. The functions being composed differ, but both constructions double the amount of computation represented at each position.



At column a , layer 2 stores b . Layer 3 therefore reads column b , whose stored answer is c , and copies c to column a . Meanwhile, column c reads e from column d . Layer 4 can now use a 's destination c to read this e . Thus the stored answer at a represents one, then two, then four lookups.

How many layers? The first two layers compute the one-step answers. Each new layer doubles the number of lookups, so reaching k lookups takes $\log_2 k$ more layers. The total depth is $2 + \log_2 k$ instead of $k + 1$.

For other values of k , keep a fixed copy of the one-step answer $f(i)$ and its token at every position i , alongside the current answer $f^s(i)$ and its token. The lookup count s is the same at every position. Both of the following operations read position $j = f^s(i)$. They differ in which answer the value map copies from that position:

operation	copy from position j	new destination at i
double	current answer $f^s(j)$	$f^{2s}(i)$
add one lookup	fixed answer $f(j)$	$f^{s+1}(i)$

For $k = 5$, the lookup counts are

$$1 \xrightarrow{\text{double}} 2 \xrightarrow{\text{double}} 4 \xrightarrow{\text{add one}} 5.$$

In the last layer, position i reads $j = f^4(i)$ and copies the fixed answer $f(j) = f^5(i)$ and its token. We keep only the one-step answer and the current answer, not all earlier powers of f . Each new answer is written into separate coordinates before the old one is cleared.

We fix k when choosing the model's weights. These weights specify which operation each layer performs. The binary digits of k give a schedule with at most $2\lceil \log_2 k \rceil$ added layers, so the depth remains $O(\log k)$. Appendix B gives this schedule, the attention maps, residual updates, and control of softmax error.

5 Broader Perspectives

There are many ways to think about the computations Transformers can perform. We have seen how attention implements lookup and composition. The literature uses related ideas to study other tasks and to prove limits on what Transformers can compute. Here are a few examples. The surveys of Liu, Makkuva, and Lee [10] and Strobl et al. [15] give a broader account.

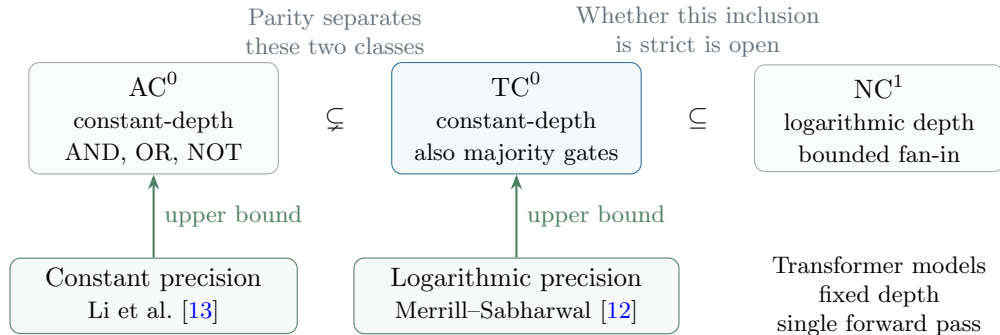
Formal languages and state machines. A formal language is a set of strings. Recognizing a string means deciding whether it belongs to that set. For a regular language, a finite-state machine can do this by updating a state as it reads each token. With state set S , token x_i defines an update $f_{x_i} : S \rightarrow S$. Starting from s_0 , the state after reading the input is

$$(f_{x_T} \circ \cdots \circ f_{x_1})(s_0).$$

Instead of following the actual state one token at a time, we can compute what each segment would do to *every* possible entering state. This is a table with $|S|$ entries. If adjacent segments have update tables g and h , their combined table maps any entering state s to $h(g(s))$. We can therefore combine neighboring segments without knowing the state that will enter them. The segments grow from length 1 to 2, then 4, then 8. After $O(\log T)$ rounds, we have the table for the whole input and can apply it to s_0 . Liu et al. [11] implement this with Transformers of logarithmic depth and width polynomial in $|S|$. Attention retrieves the neighboring table and an MLP composes the two. This bound applies to every fixed finite-state machine, but some need much less depth. For parity, uniform attention averages the bits. An MLP can use this average and the prefix length to recover the number of ones and take it modulo 2.⁴

Circuit complexity. A formal language specifies a recognition task. Circuit complexity measures the resources needed to solve it, so these views can be applied to the same problem. A Boolean circuit is a network of gates acting on input bits. Its size counts gates, and its depth counts the longest chain of gates from an input to an output. A finite-state machine processes tokens sequentially, while a shallow circuit organizes many operations in parallel. Representing a Transformer’s numbers by bits lets us compare its computation with Boolean circuits. In the logarithmic-precision model of Merrill and Sabharwal [12], fixed-depth Transformers have simulations in TC^0 : polynomial-size, constant-depth circuits with majority gates. A majority gate returns the majority bit of its inputs.

The precision assumption matters. In the arithmetic model of Li et al. [13], constant precision gives the smaller upper bound AC^0 . With a different rounding convention, Chiang [14] extends the TC^0 upper bound to polynomially many bits of precision.⁵ These are upper bounds for a single forward pass, with depth fixed as T grows. The diagram compares classes of polynomial-size circuits. Both AC^0 and TC^0 allow unbounded fan-in, meaning a gate may read a number of inputs that grows with T . Each simulation arrow uses the cited model’s arithmetic assumptions.



For example, parity belongs to TC^0 but not to AC^0 , while some regular languages are NC^1 -complete. Thus being recognizable by a finite-state machine does not automatically make a task easy for a fixed-depth Transformer: under the conjecture $TC^0 \neq NC^1$, those latter languages lie beyond the models covered by the simulation [11, 15]. Whether TC^0 is strictly smaller than NC^1 remains open.

Programs: RASP and C-RASP. RASP (Restricted Access Sequence Processing) gives us a way to write an algorithm using operations that resemble a Transformer’s: point-wise maps, selection of positions, and aggregation of their values. The following program reports the fraction of a ’s in each prefix, following Weiss et al. [16, Figure 3]. The built-in sequences `tokens` and `indices` contain the input symbols and their positions. Every line defines an operation on whole sequences, just as a Transformer layer updates all positions together.

```
is_a   = indicator(tokens == "a")
prefix = select(indices, indices, <=)
frac_a = aggregate(prefix, is_a)
```

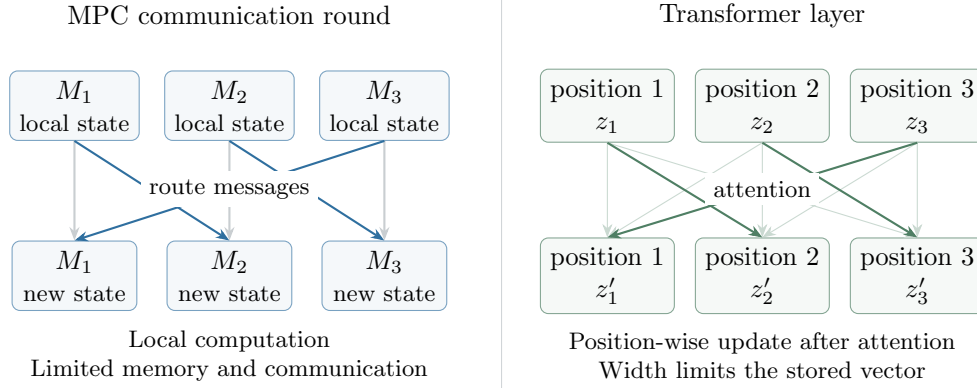
tokens	a	b	a	c	a
is_a	1	0	1	0	1
frac_a	1	1/2	2/3	1/2	3/5

The first line produces a sequence of zero–one indicators. The second calls `select(keys, queries, predicate)`: `<=` selects key position j for query position i when $j \leq i$. The third averages the selected indicators. For example, the output at position 3 is $(1 + 0 + 1)/3 = 2/3$. Composing these named operations is how we write the algorithm in RASP, before choosing weights for a Transformer.

The more restricted language C-RASP uses prefix counts directly. Counting a ’s in this input gives $(1, 1, 2, 2, 3)$. Its programs use Boolean operations, counts, addition and subtraction, and comparisons [17]. For example, comparing the counts of opening and closing parentheses tests whether a prefix has nonnegative balance. Yang and Chiang [17] show that every C-RASP program can be implemented by a causal softmax Transformer. Such programs give explicit constructions for particular algorithms. Circuit upper bounds constrain *all* choices of weights, including computations we have not programmed.

Communication and parallel computation. The INDEX argument bounds width by counting how much information attention can transmit. To understand depth, we can instead count rounds of communication. Our constructions use positions to store data, attention to exchange data, and position-wise maps to update it. The *Massively Parallel Computation* (MPC) model formalizes computation on many machines with limited memory and communication per round. Sanford et al. [9] give simulations between this model and Transformers, with overheads in memory and width. A parallel algorithm can therefore yield a Transformer construction. In the other direction, a Transformer with few layers would imply a parallel algorithm with few communication rounds.⁶

⁶The simulations use logarithmic precision and allow general position-wise maps. The general MPC-to-Transformer construction allows unrestricted attention. Simulating dense attention in MPC can require machines for pairs of positions.



This viewpoint keeps track of how much information each machine can store and exchange in a round. It therefore exposes tradeoffs between width and depth that membership in a circuit class alone does not specify. For the k -hop induction-head task, the simulations also give a conditional lower bound. Under a standard MPC conjecture, models with limited width need $\Omega(\log k)$ layers for a specified range of k [9].⁷

References

- [1] D. Hsu and A. Moitra. *How Do Transformers Work? Part II*. Simons Institute bootcamp lecture slides, September 4, 2024. [Lecture slides](#).
- [2] D. Hsu. *Transformers, Parallelism, and the Role of Depth*. NYU Math and Data Seminar, April 3, 2025. [Lecture slides](#).
- [3] N. Elhage et al. *A Mathematical Framework for Transformer Circuits*. Transformer Circuits Thread, 2021. [Research post](#).
- [4] C. Olsson et al. *In-context Learning and Induction Heads*. Transformer Circuits Thread, 2022. [Research post](#) and [paper version](#).
- [5] A. Bietti, V. Cabannes, D. Bouchacourt, H. Jégou, and L. Bottou. *Birth of a Transformer: A Memory Viewpoint*. NeurIPS, 2023. [Full paper](#).
- [6] E. Edelman, N. Tsilivis, B. L. Edelman, E. Malach, and S. Goel. *The Evolution of Statistical Induction Heads: In-Context Learning Markov Chains*. NeurIPS, 2024. [Full paper](#).
- [7] B. L. Edelman, S. Goel, S. Kakade, and C. Zhang. *Inductive Biases and Variable Creation in Self-Attention Mechanisms*. ICML, 2022. Preprint, 2021. [Full paper](#).
- [8] C. Sanford, D. Hsu, and M. Telgarsky. *One-layer Transformers Fail to Solve the Induction Heads Task*. arXiv:2408.14332, 2024. [Full paper](#).
- [9] C. Sanford, D. Hsu, and M. Telgarsky. *Transformers, Parallel Computation, and Logarithmic Depth*. ICML, 2024. [Full paper](#).
- [10] B. Liu, A. V. Makkuva, and J. D. Lee. *Sandbox for the Blackbox: How LLMs Learn Structured Data*. Draft, accessed September 23, 2026. [Survey draft](#).

⁷The conjecture says that distinguishing one cycle from two disjoint cycles requires logarithmically many communication rounds when each machine has sublinear memory. Corollary 4.3 takes even $k = \Theta(T^\xi)$, $0 < \xi \leq 1/2$, a vocabulary growing polynomially with T , $Hd = O(k^{1-\varepsilon})$ for fixed $\varepsilon > 0$, and $O(\log T)$ precision.

- [11] B. Liu, J. T. Ash, S. Goel, A. Krishnamurthy, and C. Zhang. *Transformers Learn Shortcuts to Automata*. ICLR, 2023. [Full paper](#).
- [12] W. Merrill and A. Sabharwal. *The Parallelism Tradeoff: Limitations of Log-Precision Transformers*. Transactions of the Association for Computational Linguistics, 2023. [Full paper](#).
- [13] Z. Li, H. Liu, D. Zhou, and T. Ma. *Chain of Thought Empowers Transformers to Solve Inherently Serial Problems*. ICLR, 2024. [Full paper](#).
- [14] D. Chiang. *Transformers in Uniform TC⁰*. Transactions on Machine Learning Research, 2025. [Full paper](#).
- [15] L. Strobl, W. Merrill, G. Weiss, D. Chiang, and D. Angluin. *What Formal Languages Can Transformers Express? A Survey*. Transactions of the Association for Computational Linguistics, 12:543–561, 2024. [Full paper](#).
- [16] G. Weiss, Y. Goldberg, and E. Yahav. *Thinking Like Transformers*. ICML, 2021. [Full paper](#).
- [17] A. Yang and D. Chiang. *Counting Like Transformers: Compiling Temporal Counting Logic Into Softmax Transformers*. COLM, 2024. [Full paper](#).

A Embeddings and Residual Updates

Section 2 describes what the two heads copy. Here we specify the linear maps and show how the residual connections preserve the information needed by the next layer. Let $v = |\mathcal{V}| + 1$, including $\perp$, and set $d = 2v + 3$. At position i , the initial representation is

$$z_i^{(0)} = (h_i^{\text{cur}}, h_i^{\text{prev}}, 1, i, i^2) = (e_{x_i}, 0, 1, i, i^2).$$

The token embedding is $(e_{x_i}, 0, 0, 0, 0)$ and the position embedding is $(0, 0, 1, i, i^2)$. Each block uses one head with $d_{\text{head}} = d$, followed by a ReLU MLP. We omit layer normalization. Write $[u]$ for u padded with zeros to length d . All maps below are shared across positions. Their right-hand sides are linear combinations of coordinates already stored in z_i .

First block: preserve the token and copy its predecessor. Set $\beta = \log(16T)$ and choose

$$\begin{aligned} W_Q^{(1)} z_i &= \sqrt{d} \beta [(2i - 2, -1)], \\ W_K^{(1)} z_j &= [(j, j^2)], \quad W_V^{(1)} z_j = [h_j^{\text{cur}}]. \end{aligned}$$

The constant coordinate supplies the offsets -2 and -1 . After division by $\sqrt{d}$, the score is $\beta((i - 1)^2 - (j - (i - 1))^2)$. For $i \geq 1$, its unique maximum is at $j = i - 1$, with gap at least β . Position 0 can attend only to itself.

Let o_i denote the first v coordinates of the attention output. The output projection is

$$W_O^{(1)} [o_i] = (0, o_i, 0, 0, 0).$$

Since h_i^{prev} was zero, residual addition gives $(e_{x_i}, o_i, 1, i, i^2)$. Each coordinate of o_i is within $1/16$ of the desired one-hot predecessor. Apply the coordinatewise function

$$R(t) = 2 \text{ReLU}(t - 1/4) - 2 \text{ReLU}(t - 3/4).$$

It is 0 for $t \leq 1/4$ and 1 for $t \geq 3/4$. Choose

$$\text{FFN}^{(1)}(z) = (0, R(h^{\text{prev}}) - h^{\text{prev}}, 0, 0, 0).$$

The subtraction cancels the approximate copy when the MLP output is added to the residual. The resulting state is exactly

$$z_i^{(1)} = (e_{x_i}, e_{x_{i-1}}, 1, i, i^2) \quad (i \geq 1), \quad z_0^{(1)} = (e_\perp, e_\perp, 1, 0, 0).$$

This MLP uses only affine maps and ReLUs. Linear terms can also be written with ReLUs using $t = \text{ReLU}(t) - \text{ReLU}(-t)$.

Second block: retrieve the answer. Choose

$$\begin{aligned} W_Q^{(2)} z_i &= \sqrt{d} \beta [((T+1)h_i^{\text{cur}}, 1, T+1)], \\ W_K^{(2)} z_j &= [(h_j^{\text{prev}}, j, h_{j,\perp}^{\text{cur}})], \\ W_V^{(2)} z_j &= [h_j^{\text{cur}}]. \end{aligned}$$

These maps give the match, recency, and default scores from Section 2. The query’s last two entries use the constant coordinate. The key uses the source’s position and its $\perp$ coordinate. The value copies the source’s current token. Thus o_i is within $1/16$ of the one-hot answer e_w .

We now want to replace the current token with the answer. Writing o_i directly into h_i^{cur} would add the two vectors. Instead, choose

$$W_O^{(2)}[o_i] = (2o_i, 0, 0, 0, 0).$$

After residual addition, the answer coordinate in $h_i^{\text{cur}} = e_{x_i} + 2o_i$ is at least $15/8$. Every other coordinate is at most $9/8$, even if it contained the original token. Therefore the function

$$H(t) = 2 \text{ReLU}(t - 5/4) - 2 \text{ReLU}(t - 7/4)$$

returns 1 on the answer coordinate and 0 on all others. Use

$$\text{FFN}^{(2)}(z) = (H(h^{\text{cur}}) - h^{\text{cur}}, -h^{\text{prev}}, 0, 0, 0).$$

After residual addition, the state is $(e_w, 0, 1, i, i^2)$. The first token vector now stores the answer, and the second is available for another copy. Neither block changes the position.

Finite precision. The calculations above use exact arithmetic. To evaluate softmax stably, subtract the largest scaled score before exponentiating. The denominator is then at least 1. There are at most $T+1$ terms, so exponential errors of at most c/T , for a sufficiently small constant c , give a small constant error in the normalized weights. Evaluate the scores and weighted sums accurately enough that numerical error adds at most $1/32$ to each copied coordinate. The total error is then at most $3/32$, which stays strictly inside the rounding intervals of both R and H . The stored positions, squared positions, and scaled scores have polynomial magnitude in T . Their evaluation to this accuracy requires $O(\log T)$ bits per coordinate. For a fixed vocabulary, both the residual width and the MLP width are constant in T .

B A Construction for Repeated Lookups

We now implement the composition from Section 4. The one-step destination is

$$f(i) = \max(\{j : 2 \leq j \leq i, x_{j-1} = x_i\} \cup \{0\}) \quad (i \geq 1), \quad f(0) = 0.$$

After k lookups, the answer is $x_{f^k(i)}$. Every destination is at most the position that reads it, so all lookups are causal. Position 0 stores $\perp$ and points to itself. Once a lookup reaches it, later lookups still return $\perp$.

Store the destination along with its token. Add two position coordinates P, Q :

$$z_i^{(0)} = (e_{x_i}, 0, 1, i, i^2, 0, 0), \quad z_i = (h_i^{\text{cur}}, h_i^{\text{prev}}, 1, i, i^2, P_i, Q_i).$$

Use width $d = 2v + 5$ and pad the projections to this width. The first block is unchanged. In the second block, use $W_V^{(2)} z_j = [(h_j^{\text{cur}}, j)]$, so attention copies both the answer token and its position. The output projection writes twice the token vector into h^{cur} and the position into P . The MLP rounds h^{cur} and clears h^{prev} as before. Consequently, (h_i^{cur}, P_i) stores the one-step answer and its approximate destination, while $(h_i^{\text{prev}}, Q_i) = (0, 0)$. We choose the softmax scale below to control the position error.

Compose the stored answers. Suppose every position stores its answer after s lookups:

$$h_i^{\text{cur}} \approx e_{x_{f^s(i)}}, \quad P_i \approx f^s(i), \quad (h_i^{\text{prev}}, Q_i) = (0, 0).$$

Choose

$$W_Q z_i = \sqrt{d} \beta_{\text{hop}} [(2P_i, -1)], \\ W_K z_j = [(j, j^2)], \quad W_V z_j = [(h_j^{\text{cur}}, P_j)].$$

The unscaled score is $2P_i j - j^2 = P_i^2 - (j - P_i)^2$. It selects the position nearest P_i , which is $a = f^s(i)$ when the stored position is accurate enough. That position stores its own answer after s lookups:

$$(h_a^{\text{cur}}, P_a) \approx (e_{x_{f^s(a)}}, f^s(a)) = (e_{x_{f^{2s}(i)}}, f^{2s}(i)).$$

This is why one read doubles the number of lookups.

Write the copied token and position into (h_i^{prev}, Q_i) . These coordinates were zero, so residual addition preserves the new pair there. Clear the old pair with

$$\text{FFN}(z) = (-h^{\text{cur}}, 0, 0, 0, 0, -P, 0).$$

At the next layer, read from (h^{prev}, Q) and write into the now-zero pair (h^{cur}, P) . The layer's maps specify which pair is used. No permutation of the stored coordinates is needed. We alternate between these two pairs throughout the computation.

How accurately must we store a position? Let $P_i = a + \delta$, where $a = f^s(i)$ is an integer. For any other integer position j , the unscaled score gap is

$$(2P_i a - a^2) - (2P_i j - j^2) = (j - a)^2 - 2\delta(j - a) \geq 1 - 2|\delta|.$$

The last inequality uses $|j - a| \geq 1$. Thus an error of at most $1/16$ leaves a score gap of at least $7/8$. We only need to maintain this accuracy, rather than recover an exact integer after every layer.

For $k = 2^L$, set

$$\varepsilon = \frac{1}{64T(L+1)}, \quad \beta_{\text{init}} = \beta_{\text{hop}} = 2 \log(T/\varepsilon).$$

Use β_{init} in the second initialization head. Its score gap is at least 1, so at most ε attention goes to unwanted positions. The initial position error is at most $T\varepsilon$, and the MLP makes the token vector exact. Each composition layer also assigns at most ε attention to unwanted positions, provided the position error stays below $1/16$.

To check that it does, let E_ℓ be the largest position error after ℓ composition layers. Every source already has error at most E_ℓ . The unwanted attention can mix in destinations that differ from the correct one by at most T . Therefore

$$E_0 \leq T\varepsilon, \quad E_{\ell+1} \leq E_\ell + T\varepsilon, \quad E_L \leq (L+1)T\varepsilon = \frac{1}{64}.$$

This bound is below $1/16$, so it justifies the score gap at every layer. The same argument for token coordinates uses range 1 instead of T . Their final error is at most $L\varepsilon$, so the largest coordinate gives the correct token.

Finite-precision evaluation can use part of the remaining margin. Keep the numerical error in each unscaled score below $1/16$. The computed gap is then at least $3/4$, for which the chosen β_{hop} still suffices. Limit additional error in each copied position to $T\varepsilon$ and in each token coordinate to ε per layer. The bounds above at most double, so positions remain within $1/16$ and tokens remain distinguishable. This requires $O(\log T + \log(L+1))$ bits per coordinate. For $k = 2^L$, the depth is $2 + L$, with one head per layer and residual width $2v + 5$.

A schedule for any positive integer k . To add one lookup as well as double, keep a fixed copy of the one-step answer at each position:

$$h_i^{\text{one}} = e_{x_{f(i)}}, \quad P_i^{\text{one}} \approx f(i).$$

These extra coordinates start at zero. During initialization, the second head writes the copied position into both P and P^{one} . Its MLP writes $H(h^{\text{cur}})$ into h^{one} as it rounds h^{cur} . Every later block outputs zero in the extra coordinates, so the fixed copy is preserved.

If the current destination is $P_i \approx f^s(i)$, an add-one layer uses the same query and key as a doubling layer, with value map

$$W_V z_j = [(h_j^{\text{one}}, P_j^{\text{one}})].$$

It reads position $a = f^s(i)$ and copies $f(a) = f^{s+1}(i)$ and its token. Write the result into the unused pair and clear the old pair, exactly as above. Thus each position stores only its fixed one-step answer, its current answer, and space for the next copy. The residual width is $3v + 6$.

Choose the schedule from the binary digits of k . The leading 1 corresponds to the initialized answer $f(i)$. Read each remaining digit b from left to right. First double s to $2s$. If $b = 1$, add one lookup in a separate layer, obtaining $2s + 1$. For example, $7 = (111)_2$ gives

$$1 \xrightarrow{\text{double}} 2 \xrightarrow{\text{add one}} 3 \xrightarrow{\text{double}} 6 \xrightarrow{\text{add one}} 7.$$

The weights are chosen for a fixed k , so the operation at each layer is specified in advance and is the same at every position. There are $\lfloor \log_2 k \rfloor$ remaining digits and at most two layers per digit. The total depth is at most $2 + 2\lfloor \log_2 k \rfloor$.

For the error bound, let L now be the total number of added lookup layers. An add-one layer reads the fixed answers, whose position error is only the initialization error. Its error is therefore no larger than the bound used for a doubling layer. The same choice of ε and precision suffices.